Return a Response Directly

When you create a **FastAPI** *path operation* you can normally return any data from it: a `dict`, a `list`, a Pydantic model, a database model, etc.

If you declare a Response Model FastAPI will use it to serialize the data to JSON, using Pydantic.

If you don't declare a response model, FastAPI will use the `jsonable_encoder` explained in JSON Compatible Encoder and put it in a `JSONResponse`.

You could also create a `JSONResponse` directly and return it.

> **Tip**
>
> You will normally have much better performance using a Response Model than returning a `JSONResponse` directly, as that way it serializes the data using Pydantic, in Rust.

Return a `Response`

You can return a `Response` or any sub-class of it.

> **Info**
>
> `JSONResponse` itself is a sub-class of `Response`.

And when you return a `Response`, **FastAPI** will pass it directly.

It won't do any data conversion with Pydantic models, it won't convert the contents to any type, etc.

This gives you a lot of **flexibility**. You can return any data type, override any data declaration or validation, etc.

It also gives you a lot of **responsibility**. You have to make sure that the data you return is correct, in the correct format, that it can be serialized, etc.

Using the `jsonable_encoder` in a `Response`

Because **FastAPI** doesn't make any changes to a `Response` you return, you have to make sure its contents are ready for it.

For example, you cannot put a Pydantic model in a `JSONResponse` without first converting it to a `dict` with all the data types (like `datetime`, `UUID`, etc) converted to JSON-compatible types.

For those cases, you can use the `jsonable_encoder` to convert your data before passing it to a response:

Python 3.10+

```python
from datetime import datetime

from fastapi import FastAPI
```

```python
from fastapi.encoders import jsonable_encoder
from fastapi.responses import JSONResponse
from pydantic import BaseModel


class Item(BaseModel):
    title: str
    timestamp: datetime
    description: str | None = None


app = FastAPI()


@app.put("/items/{id}")
def update_item(id: str, item: Item):
    json_compatible_item_data = jsonable_encoder(item)
    return JSONResponse(content=json_compatible_item_data)
```

Technical Details

You could also use `from starlette.responses import JSONResponse`.

FastAPI provides the same `starlette.responses` as `fastapi.responses` just as a convenience for you, the developer. But most of the available responses come directly from Starlette.

Returning a custom `Response`

The example above shows all the parts you need, but it's not very useful yet, as you could have just returned the `item` directly, and **FastAPI** would put it in a `JSONResponse` for you, converting it to a `dict`, etc. All that by default.

Now, let's see how you could use that to return a custom response.

Let's say that you want to return an XML response.

You could put your XML content in a string, put that in a `Response`, and return it:

Python 3.10+

```python
from fastapi import FastAPI, Response

app = FastAPI()


@app.get("/legacy/")
def get_legacy_data():
    data = """<?xml version="1.0"?>
    <shampoo>
    <Header>
        Apply shampoo here.
    </Header>
    <Body>
        You'll have to use soap here.
    </Body>
    </shampoo>
    """
    return Response(content=data, media_type="application/xml")
```

How a Response Model Works

When you declare a Response Model - Return Type in a path operation, **FastAPI** will use it to serialize the data to JSON, using Pydantic.

Python 3.10+

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()


class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None
    tags: list[str] = []


@app.post("/items/")
async def create_item(item: Item) -> Item:
    return item


@app.get("/items/")
async def read_items() -> list[Item]:
    return [
        Item(name="Portal Gun", price=42.0),
        Item(name="Plumbus", price=32.0),
    ]
```

As that will happen on the Rust side, the performance will be much better than if it was done with regular Python and the `JSONResponse` class.

When using a `response_model` or return type, FastAPI won't use the `jsonable_encoder` to convert the data (which would be slower) nor the `JSONResponse` class.

Instead it takes the JSON bytes generated with Pydantic using the response model (or return type) and returns a `Response` with the right media type for JSON directly (`application/json`).

Notes

When you return a `Response` directly its data is not validated, converted (serialized), or documented automatically.

But you can still document it as described in Additional Responses in OpenAPI.

You can see in later sections how to use/declare these custom `Response`s while still having automatic data conversion, documentation, etc.